

DA6401W - Assignment 3

Submitted by: Sohang Chopra <DA25M622>

Problem 2: Gradient Accumulation and Memory Constraints in Deep Learning

Training modern neural networks often requires large batch sizes for stable gradients and faster convergence. However, GPU memory limits often make large batches infeasible. In this problem, we will analyze optimizer memory usage and derive why *gradient accumulation* is used in frameworks such as PyTorch.

Assume:

- A neural network with P parameters
- Each parameter stored in 32-bit floating point (4 bytes)
- A batch size of B
- Gradient accumulation steps or number of mini-batches K .

Memory Footprint of Optimizers

Consider the memory required to store parameters, gradients, and optimizer states.

1. For **Stochastic Gradient Descent (SGD)** without momentum, determine the total memory required for storing parameters and gradients.
2. For **SGD with Momentum**, the optimizer additionally stores a momentum vector of the same size as the parameters. Compute the total memory requirement.
3. For the **Adam optimizer**, two additional vectors m_t and v_t are stored for each parameter. Compute the total optimizer memory footprint.
4. Suppose $P = 10^9$ parameters. Compute the memory required (in GB) for each of the above optimizers.

Batch Size and Activation Memory

During training, memory is also required to store intermediate activations for backpropagation. Assume the activation memory per sample is A bytes.

1. Express the total activation memory required for batch size B .
2. Suppose the GPU memory limit is M bytes. Write an inequality involving B , A , and optimizer memory that must hold for training to fit into memory.
3. Explain why increasing batch size B may become infeasible for very large models.

Convergence vs Batch Size

Empirical studies suggest that very small batch sizes produce noisy gradient estimates and slow convergence.

1. Suppose we desire an effective batch size B_{large} , but due to memory limits we can only fit a mini-batch of size b where $b < B_{large}$.

2. Propose a strategy that allows us to simulate a larger batch using multiple smaller mini-batches without increasing memory usage.
3. Let K denote the number of such mini-batches processed before updating the parameters. Express the relationship between B_{large} , b and K .

Equivalence of Gradient Accumulation and Large Batch Updates

Let the loss over a batch be defined as the average loss:

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B l(x_i, \theta)$$

where $l(x_i, \theta)$ is the loss for sample i .

Assume we divide the batch into K mini-batches each of size $b = B/K$.

1. Write the gradient of the full batch loss $\nabla_{\theta} L(\theta)$.
2. Let g_k be the gradient computed from mini-batch k :

$$g_k = \frac{1}{b} \sum_{i \in B_k} \nabla_{\theta} l(x_i, \theta)$$

Show that:

$$\nabla_{\theta} L(x_i, \theta) = \frac{1}{K} \sum_{k=1}^K g_k$$

3. Explain why gradient accumulation allows training with large effective batch sizes even when GPU memory cannot hold the entire batch simultaneously.

Solution 2

Memory-Footprint of Optimizers:

1. SGD without Momentum: memory to store parameters is P , gradients is P , so total is $2P$.
2. SGD with momentum: Additional momentum vector of same size as params is P , so total is $3P$.
3. Adam optimizer: m_t , v_t are also same size as gradients, so they are $2P$ and total is $5P$.
4. $P = 10^9$, each param is float (4 bytes), so total is 20 GB.

Batch Size and Activation Memory:

1. Total activation memory for batch is AB .
2. $AB \leq M$
3. Increasing batch size too much is infeasible due to memory limits of GPU.

Convergence vs Batch Size: Divide large batch B_{large} into smaller mini-batches of size b , calculate & accumulate gradients using each mini-batch, and only update weights using gradients once all those smaller mini-batches are done. So $B_{large} = bK$.

Equivalence of Gradient Accumulation and large Batch Updates:

1. Gradient for full batch is just average of gradients for each sample: $\nabla_{\theta} L(\theta) = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} l(x_i, \theta)$.
2. $\nabla L(\theta) = \frac{1}{B} \sum_{i=1}^B \nabla l(x_i, \theta) = \frac{1}{bK} \sum_{k=1}^K \sum_{i \in B_k} \nabla l(x_i, \theta) = \frac{1}{K} \sum_{k=1}^K g_k$
3. Even when whole large batch cannot fit into GPU memory, gradient accumulation of smaller mini-batches allows us to do equivalent weight update using above result.

Problem 3

Consider a two-layer neural network defined as follows for a single training example $(x[i], y[i])$ (square brackets denote vector indexing):

$$\begin{aligned} z_1[i] &= W_1 x[i] + b_1 \\ a_1[i] &= \text{ReLU}(z_1[i]) \\ z_2[i] &= W_2 a_1[i] + b_2 \\ y_{\text{predict}}[i] &= \sigma(z_2[i]) \end{aligned}$$

The binary cross-entropy loss is given by

$$L[i] = y[i] \ln(y_{\text{predict}}[i]) + (1 - y[i]) \ln(1 - y_{\text{predict}}[i])$$

and the empirical risk over m samples is

$$J = \frac{-1}{m} \sum_{i=1}^m L[i]$$

Here, $x[i]$ represents a single input example, and is of shape $\mathbb{R}^{D_x \times 1}$. $y[i] \in \mathbb{R}$ is single output label and is a scalar. There are m samples in the dataset. The hidden layer z_1 has D_{a_1} neurons.

1. What are the shapes of W_1, b_1, W_2, b_2 for a single example? If the network is vectorized over m examples, what are the shapes of the parameters? What are the shapes of x, y after vectorization?
2. Compute $\frac{\partial J}{\partial y_{\text{predict}}[i]}$. Refer to this quantity as $\delta_1[i]$. What is $\frac{\partial J}{\partial y_{\text{predict}}}$?
3. Compute $\frac{\partial y_{\text{predict}}[i]}{\partial z_2}$. Refer to this quantity as $\delta_2[i]$.
4. Compute $\frac{\partial z_2}{\partial a_1}$. Refer to this quantity as $\delta_3[i]$.
5. Compute $\frac{\partial a_1}{\partial z_1}$. Refer to this quantity as $\delta_4[i]$.
6. Compute $\frac{\partial z_1}{\partial W_1}$. Refer to this quantity as $\delta_5[i]$.
7. Compute $\frac{\partial J}{\partial W_1}$. Carefully indicate the shapes.

Solution 3

Rewriting formulae in terms of vectors of inputs and outputs x, y (samples are rows, features are columns):

$$\begin{aligned} z_1 &= x W_1^T + b_1 \\ a_1 &= \text{ReLU}(z_1) \\ z_2 &= a_1 W_2^T + b_2 \\ y_{\text{predict}} &= \sigma(z_2) \quad (\text{Sigmoid}) \\ J &= \frac{1}{m} \sum -y \ln(y_{\text{predict}}) - (1 - y) \ln(1 - y_{\text{predict}}) \quad (\text{Loss/Risk over all samples}) \end{aligned}$$

For vectorized network, shapes are (where m is no. of samples, d is no. of input dimensions, l_1 is no. of neurons in 1st layer (hidden)):

- $W_1 : (l_1, d), b_1 : (1, l_1)$
- $W_2 : (1, l_1), b_2 : (1, 1)$
- $x : (m, d), y : (m, 1)$

NOTE: During vector addition, numpy automatically repeats the bias vectors b_1, b_2 via array shape broadcasting (to replace 1 in bias shape dimension) to make the shapes compatible.

If passing only a single example, then $m = 1 \Rightarrow x : (1, d), y : (1, 1)$ - shapes of weights and biases aren't affected.

In below, Unit Step Function $u(z) = \text{ReLU}'(z)$ is used, which is 1 if $z > 0$ else 0:

$$\begin{aligned}\delta_1 &= \frac{\partial J}{\partial y_{\text{predict}}} = \frac{1}{m} \left(-\frac{y}{y_{\text{predict}}} + \frac{1-y}{1-y_{\text{predict}}} \right) \quad (\text{Shape: } (m, 1)) \\ \delta_2 &= \frac{\partial y_{\text{predict}}}{\partial z_2} = y_{\text{predict}}(1 - y_{\text{predict}}) \quad (\text{Sigmoid derivative; Shape: } (m, 1)) \\ \delta_3 &= \frac{\partial z_2}{\partial a_1} = W_2 \quad (\text{Shape: } (1, l_1)) \\ \delta_4 &= \frac{\partial a_1}{\partial z_1} = u(z_1) \quad (\text{ReLU derivative; Shape: } (m, l_1)) \\ \delta_5 &= \frac{\partial z_1}{\partial W_1} = x^T \quad (\text{Shape: } (d, m)) \\ \frac{\partial J}{\partial W_1} &= ((\delta_1 \cdot \delta_2) \delta_3 \cdot \delta_4)^T x = \frac{1}{m} [((y_{\text{predict}} - y) W_2) \cdot u(z_1)]^T \quad (\text{Shape: } (l_1, d))\end{aligned}$$

NOTE: Here $x \cdot y = x^T y$ is dot product.

Problem 5: Theory: Spectral Convergence of Optimization Methods

Let $f(w) = \frac{1}{2} w^T A w$ where $A \in \mathbb{R}^{d \times d}$ is symmetric positive definite with eigenvalues $\lambda_1 \leq \dots \leq \lambda_d$.

Let the spectral decomposition of A be $A = Q D Q^T$ where $D = \text{diag}(\lambda_1, \dots, \lambda_d)$.

1. Show that Gradient Descent with optimal fixed step size

$$\eta^* = \frac{2}{\lambda_1 + \lambda_d}$$

has convergence rate

$$\|w_k - w^*\|_2 \leq \left(\frac{K-1}{K+1} \right)^K \|w_0 - w^*\|_2$$

where $K = \frac{\lambda_d}{\lambda_1}$

Hint:

- Compute $\nabla f(w)$ and write the Gradient Descent update.
- Express the error iteration using $e_k = w_k - w^*$.

- Use the spectral decomposition $A = QDQ^T$ to analyze the behavior along eigendirections.

2. Explain mathematically why (using spectral arguments):

- Newton's method converges in one step.
- Conjugate Gradient converges in at most d steps.
- AdaGrad cannot eliminate dependence on K completely.

Hint:

- Compute the gradient and Hessian of $f(w)$.
- Write the Newton update and simplify using properties of A .
- Consider how optimization methods behave along the eigenvectors of A .
- Recall that Conjugate Gradient constructs A -conjugate directions.

Solution 5

1.

At optimal params w^* :

$$\begin{aligned}\nabla f(w^*) &= Aw^* = 0 \\ \Rightarrow w^* &= 0 \quad (\text{since } A \text{ is positive definite})\end{aligned}$$

Gradient Descent:

$$\begin{aligned}w_{k+1} &= w_k - \eta \nabla f(w_k) \\ \Rightarrow w_{k+1} &= w_k - \eta Aw_k \\ \Rightarrow w_{k+1} &= (I - \eta A)w_k \\ \Rightarrow w_{k+1} &= (I - \eta QDQ^T)w_k\end{aligned}$$

Error $e_k = w_{k+1} - w^* = w_{k+1} - 0 = w_{k+1}$, so:

$$\begin{aligned}e_{k+1} &= (I - \eta QDQ^T)e_k \\ \Rightarrow e_{k+1} &= Q(I - \eta D)Q^T e_k \quad (\text{since symmetric } QQ^T = I \Rightarrow QIQ^T = I)\end{aligned}$$

Let $v_k = Q^T e_k$ be error vector represented in eigen-bases of A . Then $v_{k+1} = (I - \eta D)v_k$.

But $D = \text{diag}(\lambda_1, \dots, \lambda_d)$. So each dimension of v_k is independent: $v_{k+1,i} = (1 - \eta\lambda_i)v_{k,i}$.

Contraction factor $1 - \eta\lambda_i$ is maximized at $1 - \eta\lambda_1$ (since $\lambda_1 \leq \dots \leq \lambda_d$).

Substituting optimal learning rate (given) $\eta^* = \frac{2}{\lambda_1 + \lambda_d}$, Max contraction factor is:

$$\frac{\lambda_1 + \lambda_d - 2\eta\lambda_1}{\lambda_1 + \lambda_d} = \frac{\lambda_d - \lambda_1}{\lambda_1 + \lambda_d}$$

Since $K = \frac{\lambda_d}{\lambda_1}$, Max contraction factor is $\frac{K-1}{K+1}$.

After K epochs, it's raised to K power. So:

$$\|w_k - w^*\|_2 \leq \left(\frac{K-1}{K+1}\right)^K \|w_0 - w^*\|_2$$

Hence proved.

2.

- Newton Update converges in one step to optimal $w^* = 0$ because:

$$\begin{aligned}\nabla f(w) &= Aw, \quad \nabla^2 f(w) = A \quad (\text{Gradient, Hessian}) \\ w_{k+1} &= w_k - (\nabla^2 f(w_k))^{-1} \nabla f(w_k) \quad (\text{Newton-Update}) \\ \Rightarrow w_{k+1} &= w_k - A^{-1}Aw_k = w_k - w_k = 0\end{aligned}$$

- Conjugate Descent:

Here Conjugate Gradient (a Krylov subspace method) generates a sequence of iterates w_k such that:

$$w_k \in K_k(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0\}$$

where $r_0 = \nabla f(w_0) = Aw_0$.

Using the spectral decomposition $A = QDQ^T$, any power A^m can be written as QD^mQ^T . So the subspace is effectively a span of polynomials in A acting on r_0 .

By the Cayley-Hamilton theorem, there exists a polynomial $P(x)$ of degree at most d such that $P(A) = 0$.

Therefore Conjugate Gradient converges in at most d steps. Hence proved.

- AdaGrad:

In the eigen-basis $v = Q^T w$, the quadratic objective $f(w) = \frac{1}{2}w^T Aw$ decouples into d independent 1D problems:

$$f(v) = \frac{1}{2} \sum_{i=1}^d \lambda_i v_i^2$$

For each dimension i , the gradient is $g_{t,i} = \lambda_i v_{t,i}$. AdaGrad updates as:

$$v_{t+1,i} = v_{t,i} - \frac{\eta}{\sqrt{G_{t,i}} + \epsilon} g_{t,i}$$

where $G_{t,i} = \sum_{\tau=1}^t g_{\tau,i}^2$ is the sum of squared past gradients.

Substituting $g_{t,i}$, the update becomes:

$$v_{t+1,i} = v_{t,i} \left(1 - \frac{\eta \lambda_i}{\sqrt{\sum_{\tau=1}^t (\lambda_i v_{\tau,i})^2 + \epsilon}} \right)$$

Newton's method eliminates $K = \lambda_d/\lambda_1$ by scaling exactly by $1/\lambda_i$ (the inverse Hessian). AdaGrad scales by $1/\sqrt{G_{t,i}}$. Since $G_{t,i}$ is a function of λ_i^2 , the term $\frac{\lambda_i}{\sqrt{G_{t,i}}}$ approaches a constant scale, theoretically reducing the impact of the condition number.

Because $G_{t,i}$ accumulates over time, the effective learning rate decays at a rate of $1/\sqrt{t}$. In directions with very small λ_1 , the gradient magnitude is tiny, meaning $G_{t,1}$ stays small for many iterations.

So AdaGrad cannot eliminate dependence on K completely. Hence proved.

Problem 7: Solve by hand

You are training a 3-layer neural network for a regression task. The network consists of an input layer (size 3), two hidden layers (size 3 each), and a linear output layer (size 1).

Network Architecture:

- Layer 1 (Hidden): 3 nodes. Activation: ReLU.
- Layer 2 (Hidden): 3 nodes. Activation: Sigmoid.
- Layer 3 (Output): 1 node. Activation: Linear (None).
- Loss Function: Squared Error, defined as $L = (y_{\text{predict}} - y)^2$

Initial Values:

- Input Vector $x = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$.
- Target Label $y = 1$
- Learning Rate $\eta = 0.01$

Initial Weights & Biases:(Note: Biases B_1, B_2, B_3 are all initialized to zero vectors).

$$W_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$
$$W_2 = \begin{bmatrix} -1 & 0 & 0.5 \\ 1 & 0 & -0.5 \\ 2 & 0 & -1 \end{bmatrix}$$
$$W_3 = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix}$$

Your Task:

1. Perform a forward pass to compute the network's prediction y_{predict} and the Loss L .
2. Perform a backward pass to compute the gradients of the loss with respect to all weights and biases.
3. Execute one step of Gradient Descent to calculate the updated weights W_1, W_2, W_3 and biases B_1, B_2, B_3 .

Solution 7

Activations:

- $y = \text{ReLU}(z) = z$ if $z > 0$ else 0 -- its gradient is $\text{ReLU}'(z) = 1$ if $z > 0$ else 0
- $y = \text{Sigmoid}(z) = \frac{1}{1+e^{-z}}$ -- its gradient is $y(1 - y)$

NOTE: $A \odot B$ denotes element-wise multiplication of 2 vectors.

1. Forward Pass:

$$\begin{aligned}
z_1 &= W_1 x + B_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} \\
a_1 &= \text{ReLU}(z_1) = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \\
z_2 &= W_2 a_1 + B_2 = \begin{bmatrix} -1 & 0 & 0.5 \\ 1 & 0 & -0.5 \\ 2 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \\
a_2 &= \text{Sigmoid}(z_2) = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} \\
y_{\text{predict}} &= W_3 a_2 + B_3 = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} + 0 = 3.0 \\
L &= (y_{\text{predict}} - y)^2 = (3.0 - 1)^2 = 4.0
\end{aligned}$$

2. Backward Pass (gradient calculation):

Layer 3 (output layer):

$$\begin{aligned}
\frac{\partial L}{\partial y_{\text{predict}}} &= 2(y_{\text{predict}} - y) = 2(3 - 1) = 4.0 \\
\frac{\partial L}{\partial W_3} &= \frac{\partial L}{\partial y_{\text{predict}}} a_2^T = 4.0 \begin{bmatrix} 0.5 & 0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 2.0 & 2.0 & 2.0 \end{bmatrix} \\
\frac{\partial L}{\partial a_2} &= W_3^T \frac{\partial L}{\partial y_{\text{predict}}} = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} (4.0) = \begin{bmatrix} 8.0 \\ 8.0 \\ 8.0 \end{bmatrix}
\end{aligned}$$

Layer 2:

$$\begin{aligned}
\frac{\partial a_2}{\partial z_2} &= a_2 \odot (1 - a_2) = \begin{bmatrix} 0.25 \\ 0.25 \\ 0.25 \end{bmatrix} \\
\frac{\partial L}{\partial z_2} &= \frac{\partial L}{\partial a_2} \odot \frac{\partial a_2}{\partial z_2} = \begin{bmatrix} 8.0 \times 0.25 \\ 8.0 \times 0.25 \\ 8.0 \times 0.25 \end{bmatrix} = \begin{bmatrix} 2.0 \\ 2.0 \\ 2.0 \end{bmatrix} \\
\frac{\partial L}{\partial W_2} &= \frac{\partial L}{\partial z_2} a_1^T = \begin{bmatrix} 2.0 \\ 2.0 \\ 2.0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 2.0 & 0 & 4.0 \\ 2.0 & 0 & 4.0 \\ 2.0 & 0 & 4.0 \end{bmatrix} \\
\frac{\partial L}{\partial B_2} &= \frac{\partial L}{\partial z_2} = \begin{bmatrix} 2.0 \\ 2.0 \\ 2.0 \end{bmatrix} \\
\frac{\partial L}{\partial a_1} &= W_2^T \frac{\partial L}{\partial z_2} = \begin{bmatrix} -1 & 1 & 2 \\ 0 & 0 & 0 \\ 0.5 & -0.5 & -1 \end{bmatrix} \begin{bmatrix} 2.0 \\ 2.0 \\ 2.0 \end{bmatrix} = \begin{bmatrix} 4.0 \\ 0 \\ -2.0 \end{bmatrix}
\end{aligned}$$

Layer 1:

$$\frac{\partial a_1}{\partial z_1} = \text{ReLU}'(z_1) = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{aligned}\frac{\partial L}{\partial z_1} &= \frac{\partial L}{\partial a_1} \odot \frac{\partial a_1}{\partial z_1} = \begin{bmatrix} 4.0 \\ 0 \\ -2.0 \end{bmatrix} \\ \frac{\partial L}{\partial W_1} &= \frac{\partial L}{\partial z_1} x^T = \begin{bmatrix} 4.0 \\ 0 \\ -2.0 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 4.0 & 4.0 & 4.0 \\ 0 & 0 & 0 \\ -2.0 & -2.0 & -2.0 \end{bmatrix} \\ \frac{\partial L}{\partial B_1} &= \frac{\partial L}{\partial z_1} = \begin{bmatrix} 4.0 \\ 0 \\ -2.0 \end{bmatrix}\end{aligned}$$

3. Weight Updates ($\eta = 0.01$):

$$\begin{aligned}W_3 &= \begin{bmatrix} 2 & 2 & 2 \end{bmatrix} - 0.01 \begin{bmatrix} 2 & 2 & 2 \end{bmatrix} = \begin{bmatrix} 1.98 & 1.98 & 1.98 \end{bmatrix} \\ W_2 &= \begin{bmatrix} -1 & 0 & 0.5 \\ 1 & 0 & -0.5 \\ 2 & 0 & -1 \end{bmatrix} - 0.01 \begin{bmatrix} 2 & 0 & 4 \\ 2 & 0 & 4 \\ 2 & 0 & 4 \end{bmatrix} = \begin{bmatrix} -1.02 & 0 & 0.46 \\ 0.98 & 0 & -0.54 \\ 1.98 & 0 & -1.04 \end{bmatrix} \\ B_2 &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.01 \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} = \begin{bmatrix} -0.02 \\ -0.02 \\ -0.02 \end{bmatrix} \\ W_1 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 2 \end{bmatrix} - 0.01 \begin{bmatrix} 4 & 4 & 4 \\ 0 & 0 & 0 \\ -2 & -2 & -2 \end{bmatrix} = \begin{bmatrix} 0.96 & -0.04 & -0.04 \\ 0 & -1 & 0 \\ 0.02 & 0.02 & 2.02 \end{bmatrix} \\ B_1 &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.01 \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix} = \begin{bmatrix} -0.04 \\ 0 \\ 0.02 \end{bmatrix}\end{aligned}$$

Problem 9: Numerical: Two Steps of Adam Optimizer

Consider minimizing the scalar function:

$$f(w) = \frac{1}{2}w^2$$

Consider:

- w_t : parameter at iteration t
- g_t : gradient at iteration t
- β_1 : exponential decay rate for first moment
- β_2 : exponential decay rate for second moment
- η : learning rate
- ϵ : small constant for numerical stability

Let:

$$w_0 = 2, \quad \eta = 0.1, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 0$$

1. Compute w_1 weight after 1st iteration.
2. Compute w_2 weight after 2nd iteration.

Solution 9

Adam update formulae:

$$\begin{aligned}
m_n &= \beta_1 m_{n-1} + (1 - \beta_1) g_n, & k_n &= m_n / (1 - \beta_1^n) \quad (\text{first moment}) \\
v_n &= \beta_2 v_{n-1} + (1 - \beta_2) g_n^2, & s_n &= v_n / (1 - \beta_2^n) \quad (\text{non-central second moment}) \\
w_{n+1} &= w_n - \frac{\eta k_n}{\sqrt{s_n} + \epsilon} \quad (\epsilon \text{ to avoid division by 0})
\end{aligned}$$

First Iteration:

$$\begin{aligned}
g_1 &= w_0 = 2 \\
m_1 &= 0.9 * 0 + (1 - 0.9) * 2 = 0.2 & k_1 &= 0.2 / (1 - 0.9^1) = 2 \\
v_1 &= 0.999 * 0 + (1 - 0.999) * 2^2 = 0.004 & s_1 &= 0.004 / (1 - 0.999^1) = 4 \\
w_1 &= 2 - 0.1 * 2 / (\sqrt{4} + 0) = 1.9
\end{aligned}$$

Second Iteration:

$$\begin{aligned}
g_2 &= w_1 = 1.9 \\
m_2 &= 0.9 * 0.2 + (1 - 0.9) * 1.9 = 0.37 & k_2 &= 0.37 / (1 - 0.9^2) = 1.95 \\
v_2 &= 0.999 * 0.004 + (1 - 0.999) * 1.9^2 = 0.007 & s_2 &= 0.007 / (1 - 0.999^2) = 3.5 \\
w_2 &= 1.9 - 0.1 * 1.95 / (\sqrt{3.5} + 0) = 1.795
\end{aligned}$$

Problem 11: Theoretical Question

A recommendation system predicts whether a user will **like** a movie $y = 1$ or **not like** it $y = 0$ using two features:

- $x_1 \in \{0, 1\}$: whether the movie is **fiction** (1 = fiction, 0 = non-fiction) - a **dense** feature, present for every movie. The user generally likes fiction and dislikes non-fiction.
- $x_2 \in \{0, 1\}$: whether the movie is **directed by Director Y** - a **sparse** feature, true for very few movies. Whenever Director Y directs, the user likes the movie regardless of genre.

The model is logistic regression $y_{\text{predict}} = w_1 x_1 + w_2 x_2$ with cross-entropy loss. The per-sample gradient is:

$$g_{i,t} = (\hat{y}_t - y_t) x_{i,t}$$

SGD is run for one epoch over the following 6 movies, with $y_{\text{predict}} \approx 0.5$ throughout and $\eta = 0.5$:

Sample	x_1 (fiction)	x_2 (Dir. Y)	y (liked)	Reason
1	1	0	1	fiction --> liked
2	0	0	0	non-fiction --> not liked
3	1	0	1	fiction --> liked
4	0	0	0	non-fiction --> not liked
5	1	0	1	fiction --> liked
6	0	1	1	non-fiction but Dir. Y --> liked

Observe: x_1 follows a clean pattern across all 6 samples. $x_2 = 1$ only in sample 6 - a non-fiction movie that the user likes *only* because of Director Y. This is the strongest and cleanest signal in the data, yet w_2 receives a gradient update only once in the entire epoch.

1. Compute the net weight update for each weight over the full epoch:

$$\Delta w_i = -\eta \sum_{i=1}^6 g_{i,t}$$

Show your working sample by sample for both w_1 and w_2 .

2. You should observe that w_2 receives far fewer non-zero gradient updates than w_1 across the epoch. Explain precisely why sparsity of x_2 causes learning signal scarcity of w_2 , and why a uniform learning rate η cannot compensate for this imbalance even when x_2 is the stronger predictor.
3. AdaGrad maintains a per-parameter sum of squared gradients $G_i = \sum_{t=1}^6 g_{i,t}^2$ and replaces the fixed learning rate with:

$$\eta_i^{eff} = \frac{\eta}{G_i + \epsilon}$$

Using your gradients from part 1, compute G_1 , G_2 , and the ratio $\eta_2^{eff}/\eta_1^{eff}$ (use $\epsilon = 10^{-8}$). Explain how this ratio reflects AdaGrad's response to gradient starvation.

Solution 11

Using $y_{predict} = 0.5$ (given):

Sample	x_1	x_2	y	$g_{1,1}$	$g_{2,1}$
1	1	0	1	-0.5	0
2	0	0	0	0	0
3	1	0	1	-0.5	0
4	0	0	0	0	0
5	1	0	1	-0.5	0
6	0	1	1	0	-0.5

So net weight updates using $\eta = 0.5$:

$$\begin{aligned}\Delta w_1 &= -0.5 * (-0.5 + 0 - 0.5 + 0 - 0.5 + 0) = 0.75 \\ \Delta w_2 &= -0.5 * (0 + 0 + 0 + 0 + 0 - 0.5) = 0.25\end{aligned}$$

2. w_2 gets updated only once, whereas w_1 gets updated 3 times. This is because x_2 is sparse (true in only one sample), so learning signal is sparse for it. Uniform learning rate cannot compensate because updating w_2 more would cause w_1 to update far more and overshoot.
- 3.

$$\begin{aligned}G_1 &= (-0.5)^2 + 0^2 + (-0.5)^2 + 0^2 + (-0.5)^2 + 0^2 = 0.75, & G_2 &= 0^2 + 0^2 + 0^2 + 0^2 + 0^2 + (-0.5)^2 = 0.25 \\ \eta_1^{eff} &= 0.5/(0.75 + 10^{-8}) = 0.67, & \eta_2^{eff} &= 0.5/(0.25 + 10^{-8}) = 2 \\ \frac{\eta_2^{eff}}{\eta_1^{eff}} &= 2/0.67 = 3\end{aligned}$$

Here due to gradient starvation in the case of x_2 , AdaGrad has increased its effective learning rate to compensate so that it learns better with fewer samples. Ratio 8 means x_2 is updated 8 times more with a single true sample than x_1 (assuming same gradients).

Problem 13: Effect of Initialization on Gradient Flow

Consider a deep neural network with 15 hidden layers using ReLU activation. All weights are initialized from $N(0, 0.01^2)$ (very small variance).

Which of the following is MOST likely to happen during the first few training epochs? Give a detailed explanation for your choice.

1. Activations explode as depth increases.
2. Activations shrink toward zero as depth increases.
3. Gradients become exactly zero for all neurons.
4. The network immediately reaches optimal performance

Solution 13

No. 2 is most likely - Activations shrink toward zero as depth increases.

Reason:

Weights are very small as initialized with mean 0, small variance. During forward pass, inputs are multiplied by weights to get outputs and then passed through ReLU. Since weights are small, activation outputs are also small. Each subsequent layer's output gets smaller and smaller as all weights are very small. So Activations shrink towards 0 as depth increases.

Problem 16: Numerical: Adam Update in Two Dimensions

Consider the function $f(w_1, w_2) = w_1^2 + 4w_2^2$.

At iteration $t = 1$, the parameter vector is $w_0 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

The Adam optimizer uses the following update rules:

$$\begin{aligned} g_t &= \nabla f(w_t) \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_{1,t}}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_{2,t}} \quad (\text{Bias Correction}) \\ w_{t+1} &= w_t - \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \eta \quad (\text{Parameter Update}) \end{aligned}$$

Given: $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Assume: $m_0 = [0, 0]$, $v_0 = [0, 0]$

Compute the updated parameter vector w_1 after one Adam update step.

Solution 16

$$\begin{aligned}g_1 &= [2w_1, 8w_2] = [2, 16] \\m_1 &= 0.9[0, 0] + (1 - 0.9)[2, 16] = [0.2, 1.6] \\v_1 &= 0.999[0, 0] + (1 - 0.999)[2, 16]^2 = [0.004, 0.256] \\m_1^* &= [0.2, 1.6]/(1 - 0.9) = [2, 16], \quad v_1^* = [0.004, 0.256]/(1 - 0.999) = [4, 256] \quad (\text{Bias Correction}) \\w_1 &= [1, 2] - 0.1[2, 16]/(\sqrt{[4, 256] + 10^{-8}}) = [1, 2] - [0.1, 0.1] = [0.9, 1.9]\end{aligned}$$